Passing on Passwords

Liran Ma
Department of Computer Science
Texas Christian University

Copyright © 2021 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 Password Cracking (Linux) **1**

2 (Optional) Password Cracking on Windows **3**

3 Public Key based Authentication (SSH) **6**

4 Host-based Authentication (SSH) **10**

5 Two-Factor Authentication (SSH) **13**

6 General Questions **15**

1 Password Cracking (Linux)

1.1 Lab Prep

Pull the Docker container configuration files from GitHub.

```
git clone https://github.com/liranmatcu/Docker_Security_Labs.git
# Use ``git pull'' if you have cloned the project previously
```

Build the image and start the container.

```
cd  Docker_Security_Labs/password_crack
docker build -t password-crack .   # There is a dot in the end
docker run -it --rm password-crack bash
```

1.1.1 Check the Dockfile for this lab and list all the users and their passwords in plaintext.

1.1.2 In your running container instance, find out which file stores the user name and which file stores the password hash values.

There are numbers in preceding the hash values in the file that stores the password hashes. These numbers have different meanings. For example:

```
$1$ == md5 ;      $5$ == sha256 ;    $6$ == sha512
```

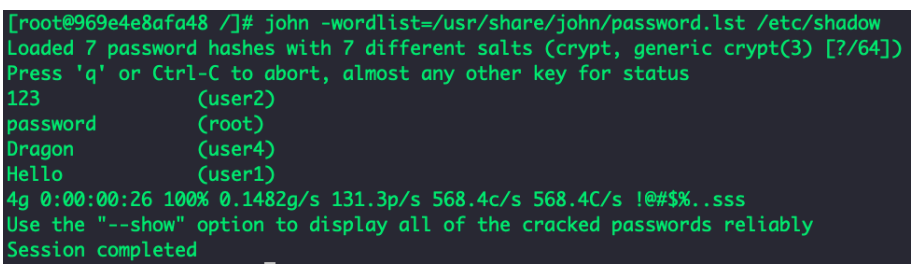
1.1.3 What are the permission differences between these two files? Check the properties of these two files using the command *ls -l filename*. Why are these two files setup this way?

1.2 Crack SHA-based password hashes

Run John the Ripper on the shadow password file.

```
john  --wordlist=/usr/share/john/password.lst  /etc/shadow
```

An example cracking output is shown in Fig. 1.



```
[root@969e4e8afa48 /]# john -wordlist=/usr/share/john/password.lst /etc/shadow
Loaded 7 password hashes with 7 different salts (crypt, generic crypt(3) [?/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
123              (user2)
password         (root)
Dragon           (user4)
Hello            (user1)
4g 0:00:00:26 100% 0.1482g/s 131.3p/s 568.4c/s 568.4C/s !@#$%..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Figure 1: Example cracking output.

1.2.1 Provide a screenshot of the john command and its results.

To display the cracked password cracking results,

```
john --show /etc/shadow
```

1.2.2 Provide a screenshot of the cracked passwords.

Enable word mangling rules for the wordlist mode as follows.

```
john  --wordlist=/usr/share/john/password.lst --rules /etc/shadow
```

Some passwords should be cracked quickly and others taking much longer. The more complicated passwords might take hours or days to crack. Press the space-bar will show the passwords being tried. Hit it several times to see the way the password combinations are tried. Stop the program after some time using Ctrl-C.

1.2.3 Provide a screenshot of the cracked passwords. Did you get more passwords cracked? Are there any passwords left uncracked? Why?

You can check the default password file that comes with John the Ripper.

```
cat /usr/share/john/password.lst | more
```

Other world list files can be downloaded from https://wiki.skullsecurity.org/Passwords.

1.2.4 (Optional) Try using the brute force mode to crack the uncracked password hashes.

John the Ripper usage examples can be found here: http://www.openwall.com/john/doc/EXAMPLES.shtml; and a cheat sheet https://countuponsecurity.files.wordpress.com/2016/09/jtr-cheat-sheet.pdf.

1.3 (Optional) Crack MD5-based password hashes

We can make the system to create users' passwords in MD5 format. On a Linux machine (or a virtual machine), type the following commands as the root user to change the hashing algorithm to MD5.

```
authconfig --test | grep hashing
authconfig --passalgo=md5 --update
authconfig --test | grep hashing
```

Create a few users (with their passwords) and repeat the steps in Section 1.2.

2 (Optional) Password Cracking on Windows

To complete this lab task, a Windows machine (or virtual machine is required). Reference: https://samsclass.info/123/proj10/px16-hashcat-win.htm.

2.1 Lab Prep

First, you need to dump Windows password hashes (LM and/or NTLM) using use an utility.

The password extracting tool Cain & Abel may be obtained here: http://csfaculty.tcu.edu/ma/cosc40053/downloads/ca_setup.zip

Download and unzip John the Ripper in Windows from here: http://csfaculty.tcu.edu/ma/cosc40053/downloads/john179w2.zip

It would be easier to place the unzipped folder on the Desktop of your Windows.

2.2 Create Test User Accounts

Create additional user accounts and their associated passwords on the Windows machine. On your Windows 7 machine, click Start. Type in CMD and press Shift+Ctrl+Enter.

If a "User Account Control" box pops up, click Yes.

In the Administrator command prompt window, execute this command:

```
net user username password /add
```

Replace the username and password with the following username and password.

[Username]	[Password]
User1	Hello
User2	123
User3	Flower
User4	Dragon
User5	Hellodragon
User6	123Hello

2.3 Extract Password Hashes

The first step to cracking passwords is to get the password hashes.

- On your Windows 7 desktop, right-click the Cain icon and click "Run as Administrator".
- In Cain, on the upper set of tabs, click Cracker.
- In Cain, move the mouse to the center of the window, over the empty white space.
- Right-click and click "Add to list...".
- In the "Add NT Hashes from" box, accept the default selections and click Next.

The password hashes appear, as shown in Fig. 2.

There are two password hashes: LM Hashes and NT hashes.

- LM hashes are very old and so weak even Microsoft has finally stopped using them by default in all Windows versions after Windows XP.

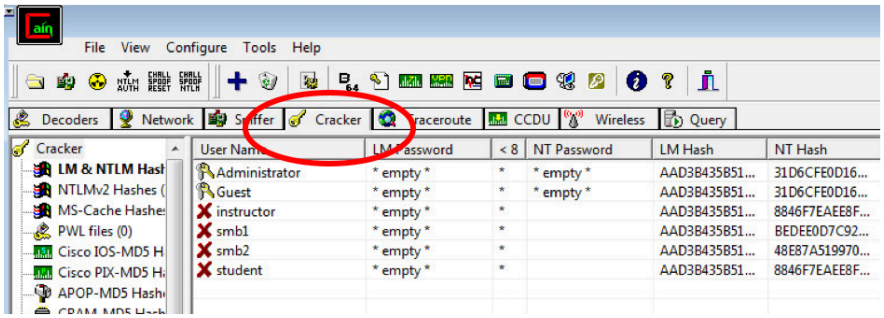


Figure 2: Example of password hashes.

- NT hashes are Microsoft's "more secure" hash, used by Windows NT in 1993 and never updated in any way. As you will see, these hashes are also very weak and easily cracked, compared with Linux password hashes.

Windows password hashes are weaker than Linux hashes. Notice that your NT password hash starts with 8846, just like mine. This is because Microsoft does not salt hashes–every user on every Windows machine has the same salt if they are using a password of password. That means you can often crack Windows password hashes by just Googling them, because many lists of common passwords and hashes have been uploaded to the Internet over the last 20 years.

2.4 Dictionary Attack

In Cain, select all the users and right click. Choose Dictionary Attack -> NTLM Hashes (Fig. 3).

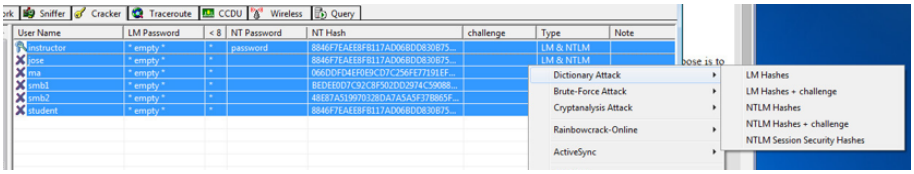


Figure 3: Choose NTLM Hashes.

The dictionary attack window will popup. You first need to add a password list (in txt format) to the dictionary attack menu. The password list can be downloaded from: http://csfaculty.tcu.edu/ma/cosc40053/downloads/password.txt

Right click the empty space under File and choose Add to list Fig. 4).

After adding the password list, uncheck some options as follows so as to save time. Click start to launch the attack Fig. 5).

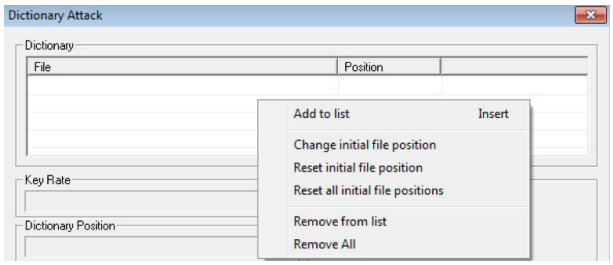


Figure 4: Adding a password list.

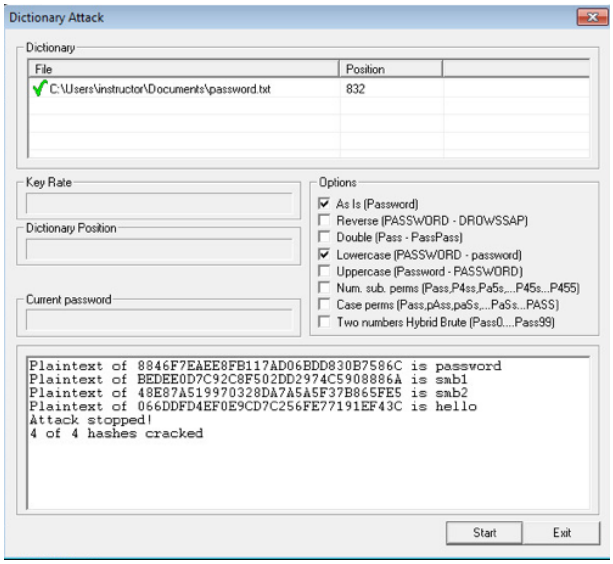


Figure 5: Launch the attack.

2.4.1 How many passwords were found?

2.4.2 What are differences between a brute-force attack, a dictionary attack, and a rainbow table attack?

2.4.3 How would you protect yourself from password cracking utilities? Provide adequate explanation.

2.5 Brute-force Attack

If you have time, you can launch a brute-force attack.

3 Public Key based Authentication (SSH)

Passwords are known to be a weak authentication method. SSH allows a client to prove its identity to a remote host by using a public keys instead of a password. In this lab, we will exercise public key based SSH authentication to avoid passwords.

3.1 Lab Prep

Download lab related files from GitHub. Go to your local GitHub folder, and pull the files using the git command as follows.

```
git pull https://github.com/liranmatcu/Docker_Security_Labs.git
# Use ``git clone'' if you have not cloned the project before
```

Build and start SSH client

```
cd  Docker_Security_Labs/authentication/ssh_client
docker build -t ssh-client .   # There is a dot in the end

docker run -it --name client --hostname ssh-client --rm
-v "$PWD"/pub_keys:/c_keys ssh-client bash
# The above two lines are one command
```

There are three users created on the client container. They are *root, student* and *instructor*; their password is password. Check the Dockerfile for more information if needed.

Build and start SSH server

Open another terminal on your machine and issue the following commands.

```
cd  Docker_Security_Labs/authentication/ssh_server
docker build -t ssh-server .    # There is a dot in the end

docker run --name server --hostname ssh-server --rm
-v "$PWD"/pub_keys:/s_keys ssh-server
# The above two lines are one command
```

There are also three users created on the server container. The users are are root, student and instructor; their password is password (i.e., they all have the same password).

3.2 RSA Key Pair Generation on Client

A RSA key pair must be generated on the **client**. The public portion of this key pair will reside on the server to be connected to, while the private portion needs to remain on a secure local location on the client. The key generation can be done with the ssh-keygen utility.

Create a 2048 bit RSA key on your client as a regular user (here, we use the **student** user as an example).

```
su - student
ssh-keygen -t rsa
```

```
Generating public/private rsa key pair.
Enter file in which to save the key (/home/student/.ssh/id_rsa): Hit enter
Created directory /home/student/.ssh
Enter passphrase (empty for no passphrase): Hit enter for no passphrase
Enter same passphrase again: Hit enter again for no passphrase
```

Note that if you supply a passphrase, the private key is encrypted by the passphrase. You will need to supply the passphrase in order to use the private key for authentication.

Your private has been saved in

```
/home/username/.ssh/id_rsa
# Here, username is student in our example

more /home/username/.ssh/id_rsa
# View the private key
```

Your public key has been saved in

```
/home/username/.ssh/id_rsa.pub
```

Transfer the */home/user/.ssh/id_rsa.pub* file to the server you want to connect to. You can use *scp* to transport the key file.

```
scp /home/username/.ssh/id_rsa.pub   username@server-IP-address:
# There is a colon (``:'') in the end of the command
```

You can get the IP address of the server by opening up a terminal on your host machine and running ***docker inspect server*** (It should be something along the lines of 172.17.0.3)

3.3 Configure Client Public Key on Server

On your server, become the user (in this example the *student* user) you want to use the public key for. Concatenate the contents of *id_rsa.pub* as follows.

If you are using a Docker container, the server will already be running without an interactive shell. So you will need to open a new terminal and run **docker exec -it server bash** to do the following

```
docker exec -it server bash

su - student
```

```
# If the authorized_keys file does not exist, create it.
mkdir /home/student/.ssh/
touch /home/student/.ssh/authorized_keys

cat /home/student/id_rsa.pub >> /home/student/.ssh/authorized_keys
chmod 600 /home/student/.ssh/authorized_keys
chmod 700 /home/student/.ssh/
```

In your ssh-server container, the sshd service has been started. You are ready to test the SSH connection from a client.

3.4 Test Public Key based Authentication

Go back to your SSH client container, as the student user, connect to the SSH server using the ssh command.

```
ssh  -i  private-key-file  username@server-IP-address
# username is student in our example
# Note that private-key-file is /home/username/.ssh/id_rsa
```

If you are NOT prompted for a password, you have successfully setup SSH public key authentication.

You can see more information about what is happening by adding the -v option.

```
ssh   --vvv  -i  private-key-file  user@server-IP-address
```

3.4.1 Provide a screenshot of your ssh connection results (showing public key).

In addition, you can place the student user's public key into another user's (e.g., instructor or root) *authorized_keys* file inside its *.ssh* directory. Test what will happen when you try to log in as another user. For example,

```
ssh  -i  private-key-file  instructor@server-IP-address
```

3.4.2 Provide a screenshot of your ssh connection results (student logs in as instructor or root).

3.4.3 What are the pros and cons of this type of authentication approach?

3.5 Elliptic Curve Digital Signature Algorithm (ECDSA)

ECDSA offers a variant of the Digital Signature Algorithm (DSA) that uses elliptic curve cryptography. ECDSA has more advantages over RSA. Its small key sizes are beneficial in environments where resources such as storage space are limited. In addition, it runs its key and signature generation algorithms much faster than RSA.

Repeat steps listed in Sections 3.2 to 3.4. When typing the commands, replace the key word **rsa** with **ecdsa** (when applicable).

3.5.1 Provide a screenshot of your ssh connection results showing ECDSA used for authentication.

4 Host-based Authentication (SSH)

Host-based authentication allows hosts to authenticate on behalf of all or some of the system's users. This type of authentication can be useful for managing computing clusters and other fairly homogeneous pools of machines. Ref: http://en.wikibooks.org/wiki/OpenSSH/Cookbook/Host-based_Authentication.

4.1 SSH Server Configuration

There are three files on the SSH server that need to be modified to enable host-based authentication.

- /etc/ssh/sshd_config - turns on host-based authentication
- /etc/ssh/shosts.equiv – contains the client addresses that are allowed to do host-based authentication
- /etc/ssh/ssh_known_hosts - holds the identities (i.e., **the host public keys**) of the clients; Note they are not public keys of a user but the host.

/etc/ssh/sshd_config: **In our Docker container (i.e., ssh-server), these changes have been made in its /etc/ssh/sshd_config file.** The following instructions are provided to you as a reference. To allow host-based authentication on the server, modify the following two options in the /etc/ssh/sshd_config file (also shown in Fig. 6).

```
HostbasedAuthentication   yes
IgnoreRhosts   no
```

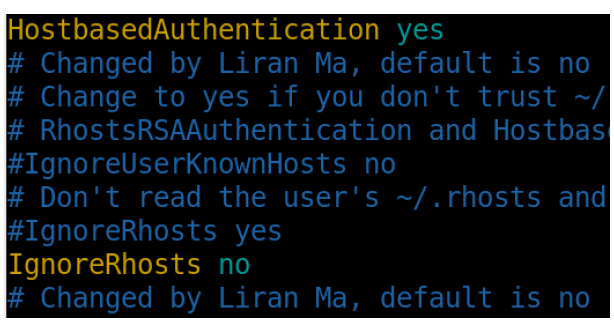


Figure 6: Enable host-based authentication.

/etc/ssh/shosts.equiv: The file should contain client's hostname and its IP address. An example is given as follows. You may need to create the file.

```
ssh-client
172.17.0.2
# Change the hostname and IP according to your actual settings
```

/etc/ssh/ssh_known_hosts: Those hosts (i.e., clients) listed in the server's shosts.equiv must also have their host public keys stored in */etc/ssh/ssh_known_hosts* on the server. You may need to create this file as well. There are three required data fields per line. An example is shown in Fig. 7. First is the host name or IP address or comma separated list of them, corresponding to those from shosts.equiv. Next is the key type, either ssh-rsa for RSA keys or ssh-dss for DSA keys. Third is the **host public key of the client** (you will need obtain to the host public key data from the client).

```
ssh-client,172.17.0.2 ssh-rsa AAAAB3N... (can be very long)
# Note that the data fields for each host-IP-key combination
# should be within a single line
```

```
root@ssh-server:~ $ cat /etc/ssh/ssh_known_hosts
ssh-client,172.17.0.2 ssh-rsa AAAAB3NzaC1yc2EAAAADAQ
8qGJuoBv0FbtY1YLRv1Vj7B/qZz0+yYZLWffCllhRbqUdfQgpa7N
nFTuUFCwzxl6gixSPow+Wt6YZxpSlMNo349iEkoq3ch+Kgh9Be42
Wbna3CelLVKc0XPVfwscyaUVa24g6OOsSCMNz40lFtgqspDPnfWt
```

Figure 7: Host and its public key in /etc/ssh/ssh_known_hosts.

The SSH client's host public keys have been created in our ssh-client container instance. If a client does not have host public keys, they need to be generated (or regenerated) using "*ssh-keygen -A*" on the client.

There are different ways of transporting the host public key data from the client. For example, it can be copied using *sftp* from the client to the server.

```
scp /etc/ssh/ssh_host_rsa_key.pub   root@server:/etc/ssh
# or
scp /etc/ssh/ssh_host_dsa_key.pub   root@server:/etc/ssh
# if you prefer to use DSA
```

Or just copy and paste it between terminals (one on client, the other on server).

After configuring the three files, we are ready to move to the client configuration. Note that the sshd service has been started in the ssh-server container instance.

4.2 SSH Client Configuration

On the SSH client, there is a single configuration file (*/etc/ssh/ssh_config*) that needs to be modified to allow host-based authentication. **In our Docker container (i.e., ssh-client), the file have**

been configured. So, you do not need to make any changes to the file. The following instructions are provided to you as a reference. Here is an excerpt from /etc/ssh/ssh_config on the client allowing host-based authentication to other machines (also shown in Fig. 8).

```
HostbasedAuthentication  yes
EnableSSHKeysign         yes
```

```
#   HostbasedAuthentication no
    HostbasedAuthentication yes
#   Changed by Liran Ma
    EnableSSHKeysign yes
#   Added by Liran Ma
```

Figure 8: Client configuration for host-based authentication.

4.3 Test Host-based Authentication

Open another terminal onto your SSH client container instance. As a regular user (e.g., student or instructor, not the root user), delete the user's public keys that you have created in previous sections.

```
docker exec -it client bash
su  -  user  # replace user with student or instructor

rm /home/user/.ssh/id_rsa*
# And/Or other keys such as id_dsa and etc.
```

Next, try to connect to the server using host-based authentication.

```
ssh -vvv ssh-server-IP-address
```

If you are prompted for a password, something is wrong. Verify that the steps have been followed correctly. Otherwise, congratulations! You have successfully setup host-based authentication.

```
debug2: userauth_hostbased: chost ssh-client.
debug3: ssh_msg_send: type 2
debug3: ssh_msg_recv entering
debug3: ssh_keysign: [child] pid=100, exec /usr/lib/ssh/ssh-keysign
debug3: send packet: type 50
debug2: we sent a hostbased packet, wait for reply
debug3: receive packet: type 52
debug1: Authentication succeeded (hostbased).
Authenticated to 172.17.0.3 ([172.17.0.3]:22).
```

Figure 9: Host-based authentication succeeded.

4.3.1 Take a screenshot showing the success of the host-based authentication.

5 Two-Factor Authentication (SSH)

Two-factor authentication (2FA) is an authentication method that requires entering more than one piece of information to successfully log in to an account or device. In addition to entering a username and password combination, 2FA requires the user to enter an additional piece of information such as a one-time password (OTP), like a six-digit verification code.

In this lab, you will need to use a **Ubuntu** virtual machine (VM) on AWS. The instructions are adopted from https://www.digitalocean.com/community/tutorials/how-to-configure-multi-factor-authentication-on-ubuntu-18-04.

5.1 Lab Prep

Log onto your AWS Ubuntu VM.

```
ssh -i "path-to-the-private-key-file(.pem)"  ubuntu@AWS_VM_IP
```

Install required software as follows.

```
sudo apt-get update
sudo apt-get install libpam-google-authenticator
```

5.2 Activate 2FA in Ubuntu

We will configure it to require 2FA only during log in.

```
sudo vim /etc/pam.d/common-session
```

Add the following highlighted line to the end of the file.

```
# end of pam-auth-update config
auth required pam_google_authenticator.so nullok
```

This will tell Ubuntu to require 2FA when a user connects to the system via the command line (either locally or remotely through SSH), but not during subsequent authentication attempts, such as sudo requests.

You have now successfully configured Ubuntu to prompt you for 2FA either just during log in, or for every authenticated action performed on the system.

In addition, verify that your AWS VM allows password authentication. Go to the "/etc/ssh/sshd_config" file on your AWS VM, change the "ChallengeResponseAuthentication" option from "no" to "yes" (if it was no).

```
ChallengeResponseAuthentication  yes
```

Then, restart the sshd service.

```
sudo systemctl restart sshd.service
```

Next, we will create a user and configure the user to be prompted for 2FA when logging in to the Ubuntu installation.

5.3 Configure 2FA for a New User

Create a user (say demo) as follows.

```
sudo adduser demo
```

Write down the password assigned for the newly created user for future use.

Run the *google-authenticator* command to start and configure the PAM module for the newly created user (again, in our example the user name is demo):

```
sudo su - demo
google-authenticator
```

The command will present a prompt that will ask you several configuration questions. Answer yes to these questions would be fine for this lab. Note that you will need to **save your new secret key, your verification code**, **your emergency scratch codes**, and **the link** to download the QR code. You can use the QR code to setup 2FA apps such as Authy (https://authy.com/).

5.4 Test 2FA

In this section, we will test 2FA by logging into your Ubuntu environment as the demo user. Try switching to the demo user from the ubuntu user using the su command.

```
exit  # Assume that you currently log in as the demo user
# The exit command will let you return to the ubuntu user

su - demo  # Try switching to the demo user
```

You will be prompted for a password. After entering the correct password, you will be prompted for a verification code. You can use one of the emergency codes as the verification code to log in for this lab, as the example shown in Fig. 10.

Moreover, you can open another terminal on your local machine. Try sshing into your Ubuntu environment as the demo user remotely as follows.

```
ssh  demo@AWS_VM_IP
```

```
TCUDT20JV40:~ lma$ ssh demo@34.238.240.60
Password:
Verification code:
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux
```

Figure 10: 2FA log in with a verification code.

Similarly, you will be prompted for a password and a verification code. You also can use one of the emergency codes as the verification code to log in (note that each code can be used only once), as the example shown in Fig. 10.

5.4.1 Provide a screenshot showing your demo user logging in with password and verification code.

5.5 (Optional) Use a 2FA app

If you have a smart device, you can install a 2FA app to generate a time-varying verification code. Install the Google Authenticator App (https://play.google.com/store/apps/details?id=com.google.android.apps.authenticator2&hl=en_US) or Authy App (https://authy.com/) on your mobile devices and use a real-time generated verification code.

6 General Questions

6.0.1 Any corrections or general comments about this lab?

6.0.2 How long did it take you to complete this lab?

6.0.3 Was it an appropriate length lab?